

架构

功能分析：

- **GUI:** ImGui (启用 Docking 分支，支持窗口拖拽停靠)
- 模型加载: tinygltf (配合 Vulkan 封装)
- 纹理加载: stb_image
- 数学库: glm (注意：需处理 Vulkan Y轴向下 vs GLM Y轴向上的差异)
- JSON库: nlohmann/json (现代C++标准库首选)
- Gizmo: 在视口支持三轴拖拽 (ImGuiizmo 库)。
- **实时编辑流** 即时模式 (IM) 哲学设计:
- 类似 Blender/Godot，选中物体即时修改属性。
- **材质系统:** 材质作为独立资源，修改材质参数 (颜色、金属度等) 优先通过 **Push Constants** 更新以实现即时反馈，纹理更换则通过 DescriptorSet。
- **音频:** 简单的 3D 音频播放，SDL库自带 (基于位置衰减)。

布局逻辑 (GUI)



架构-1769497227860.webp

GUI 每个部分都支持折叠，逻辑上我们需要划分为以下区域：

1. 顶部菜单栏 (Main Menu)
 - **文件:**
 - 新建/加载/保存场景 (JSON)
 - 导入资源 (模型/纹理/音频 -> 拷贝到 Assets 目录并生成 .meta 文件)

- **创建:**
- 基础图元 (Cube, Sphere, Plane)
- 灯光 (Point, Directional)
- 摄像机
- **调试:**
- 渲染模式 (Shaded, Wireframe, Albedo, Normal)

1. 左侧：场景层级 (Scene Hierarchy)

- 以树状结构显示所有 Node。
- 支持拖拽改变父子关系。
- 显示节点名称与显隐状态 (Active)。

3. 右侧：属性面板 (Inspector)

分上下部分

上：

顶部：

- 当前选中的物体的名称
- UUID

下：使用 纵向排列的**Tab 标签页** 切换，避免横向空间拥挤。

- Tab1：World环境（常驻）
- 其余的tab：
 - 如果选中的是模型：材质、网格、Transform
 - 如果选中的是灯光：transform、灯光属性（颜色、亮度等等）
 - 如果是....

4. 底部：资源浏览器 (Content Browser)

- 显示 Assets 文件夹下的文件，支持拖拽资源到属性面板赋值。

设计哲学

“万物皆节点，资源即引用，渲染靠提交”

1. **Node (节点)**: 场景中的具体对象，负责“在哪里(Transform)”和“层级关系(Hierarchy)”。
 2. **Resource (资源)**: 硬盘上的文件数据。资源通过 **GUID** 被节点引用，支持 AssetRegistry 自动管理路径映射。
 3. **UUID/GUID**: 每一个资源文件和每一个场景节点都有一个全球唯一ID，用于序列化引用。
 4. **Render Queue (渲染队列)**: Node 不直接执行 `vkCmdDraw`。在渲染阶段，Scene 遍历 Node 提取 `RenderPacket` 提交给 `Renderer`，由后端统一进行排序和合批录制。
-

类结构 (C++)

基础类

- **Object**: 所有类的基类，包含 `UUID` 和 `Name`。

Resource (资源基类)

继承自 `Object`。管理 Vulkan 句柄的生命周期。

- **MeshResource**: 包含顶点/索引缓冲区的 `VkBuffer`，顶点数量，AABB包围盒。
- **TextureResource**: 包含 `VkImage`，`VkImageView`，`VkSampler`。
- **MaterialResource**: 包含 Shader 引用，以及参数包 (`BaseColor`, `Metallic`, `Roughness` 等)。

Node (节点基类)

继承自 `Object`。场景图的核心。

- **状态**: `isActive`
- **层级**: `Node* parent`，`std::vector<Node*> children`
- **变换**: `glm::vec3 position`，`rotation`，`scale`；缓存 `localMatrix` 与 `globalMatrix`。
- **核心虚函数**: `CollectRenderables(SceneRenderer& renderer, const glm::mat4& parentMatrix)`：递归收集渲染信息并提交到渲染器队列。

MeshNode (原 Geometry)

继承 `Node`。

- **引用**: `UUID meshID`，`UUID materialID`。
- **渲染逻辑**: 在 `CollectRenderables` 中将 `meshID`、`materialID` 和 `globalMatrix` 打包提交给渲染器。

LightNode (灯光基类)

继承 `Node`。

- **通用参数**: `Color`，`Intensity`
后续的平行光、点光、射灯继承这个类。

CameraNode

继承 `Node`。

- **通用参数**: `IsPrimary`，`ProjectionType`
- **功能**: 提供 `GetViewMatrix()` 和 `GetProjectionMatrix()`。

Material

- `name`，`index`，`doubleSided`，`alphaMode`
- `pbrMetallicRoughness`: `baseColorFactor`，`metallicFactor`，`roughnessFactor`

- 贴图引用: `baseColorTexture`, `metallicRoughnessTexture`, `normalTexture` 等 (均通过 `UUID/index` 引用)
-

工程与场景

工程 (Project):

- 物理表现为一个包含 `project.json` 的文件夹。
- **AssetManager**: 启动时扫描 `Assets/` 目录, 通过 `.meta` 文件建立 `GUID -> 文件路径` 的映射表。
- 负责管理全局资源加载与卸载 (如单例访问 `MeshResource`)。

场景 (Scene):

- 一个工程可有多 `个 .json` 格式的场景文件。
- **层级结构**: 场景维护一个隐式的虚拟根节点 (Root Node), 所有物体都是其子孙。
- **序列化**: 使用 `nlohmann/json` 递归读写 `Node` 树及其 `UUID` 引用关系。
- **导入支持**: 支持导入 `GLB/GLTF`。导入时会解构几何与材质资源存入 `Assets` 目录, 并自动在场景中生成对应的 `MeshNode`。